

Advanced Manual **Smart Contract Audit**

September 30, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

Nlyra

Real Yield Staking v2, interfaces/External.sol

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	3	0	3	0
● Low-Risk	1	0	0	1
● Medium-Risk	0	0	0	0
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

Nlyra

Website	nlyra.xyz
---------	-----------

Blockchain	Robinhood Chain (Arbitrum Orbit, chainId 4663)
------------	--

Source Code	github.com/NlyraProject/nlyra-platform, staking-v2/src (commit 19880c0)/interfaces/External.sol (37 lines)
-------------	--

Contract Address	Not separately deployed (interface definitions only)
------------------	--

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Audit Summary	2
Audit Scope	3
Audit Method	3
Automated Vulnerability Check	3
Manual Code Review	3
Used tools	3
Table of Contents	4

● The Interfaces Are Deliberately Minimal, Which Is The Right Choice

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
EXT-I1	This file contains no logic, only the small set of functions the project needs to call on three outside systems. What stands out is that it declares just the handful actually used rather than each system's full published interface, which is the safer choice because anything listed here becomes something some part of the project could be made to call.

Code Snippet

```
/// Interfaces minimas de contratos externos (Pons, Uniswap v3, WETH).  
/// Copiadas de las fuentes verificadas; solo lo que usamos.  
  
interface IWETH9 {  
    function deposit() external payable;    // withdraw() is not declared  
}
```

Recommendation

No change is needed. Keep the file trimmed to what is used, and resist adding functions here speculatively, since anything declared becomes something the project is able to call.

● Two Declared Functions Are Never Used Anywhere In The Project

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
EXT-I2	Two of the declared fee locker functions, one reading and one changing where a token's fees are sent, are never called by any contract in this audit. Leaving them declared is harmless, since declaring a function grants no permission to call it, but it contradicts the file's own stated principle of listing only what is used and could lead a reader to assume the project controls the fee destination when it does not.

Code Snippet

```
interface IPonsLaunchLocker {
    function collectFees(address token) external returns (...); // used
    function feeRedirects(address token) external view returns (address);
    function setFeeRedirect(address token, address newFeeWallet) external;
}
// the last two are never called by any audited contract
```

Recommendation

Either remove the two unused declarations to match the file's stated principle, or add a short note saying they are kept for reference and are exercised by the token owner rather than by these contracts. Either resolves the ambiguity.

● We Confirmed The Uniswap Details Independently

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
EXT-I3	The trading pool functions declared here match the standard Uniswap version 3 definitions, and we independently checked the pool fingerprint the staking contract uses to prove a pool address is genuine: it is the published canonical value rather than a substituted one. That matters because it means the contract's pool authenticity check really does prove it is talking to a real Uniswap pool for the expected pair and fee level, so we are recording it as verified rather than assumed.

Code Snippet

```
// declared here, and matching the canonical Uniswap v3 definitions:
function swap(address recipient, bool zeroForOne, int256 amountSpecified,
              uint160 sqrtPriceLimitX96, bytes calldata data)
    external returns (int256 amount0, int256 amount1);

// and the fingerprint used to prove a pool is genuine is the
// standard published Uniswap v3 value:
// 0xe34f199b19b2b4f47f68442619d555527d244f78a3297ea89325f843f87b8b54
```

Recommendation

No change is needed. Keep the fingerprint and the factory address as fixed values, since the protection they provide depends on them not being adjustable after deployment.

● The Fee Collection Details Are Copied By Hand And Nothing Checks Them

Low-Risk - Won't cause any problems, but can be adjusted for improvement [Status: Resolved]

Error Code	Description
EXT-L1	The fee locker's collection function and the two failures the splitter is willing to tolerate were copied by hand and nothing in the project confirms the copy is accurate, so a wrong function name would have made every collection fail permanently with no way to repair it. The team has supplied a live collection that called the locker and paid the staking contract, which is exactly the confirmation this finding asked for and settles the question in practice. We record the evidence as transaction 0xd5d5d9b1717fab7eed17e6f5fde1babb3f7e03658790aa4e4b27cf7cf3b10886 in block 75,688,653. Coinsult was unable to reach the network from its own environment, so this rests on the team's attestation and can be checked by anyone on the block explorer.

Code Snippet

```
// the assumption that could not be checked from the code alone:
function collectFees(address token) external returns (...);
error NoFeesToCollect();
error NotAuthorized();

// settled in practice by a live collection supplied by the team:
// tx 0xd5d5d9b1717fab7eed17e6f5fde1babb3f7e03658790aa4e4b27cf7cf3b10886
// block 75,688,653
// (not independently fetched by Coinsult; verifiable on the explorer)
```

Recommendation

No further action needed. If the fee locker is ever replaced or upgraded at the source, treat a successful live collection as a check worth repeating, because the splitter has no owner and cannot be repaired if the interface stops matching.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.